

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 4, Section 4.1
- **Estimated reading time:** 25 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 1

MARKOV'S PROPERTY AND GBM

Derivative Pricing - Module 4



Outline

- ▶ Stochastic Processes and the Markov Property
- ▶ (Geometric) Brownian Motion (GBM)
- ▶ Continuous-Time Stochastic Process for Stock Prices

Stochastic Processes and Markov

Now that we have mastered option pricing in discrete time, let's move to a continuous-time framework.

⇒ To that end, we'll use a special type of stochastic process, a **Markov Process**:

- ▶ In a continuous-time stochastic process, the underlying variable can take any value within a range.
- ▶ In a Markov process, only the current value of the variable matters for predicting future.
⇒ i.e., Past path is irrelevant and/or current price contains all info → Weak-form efficiency
- ▶ For stock price modeling, we will use a special type of Markov: a **Wiener process**.
 - This is a process to model changes in stock price, also called Brownian Motion
 - Brownian Motion processes have a mean change of 0 and a variance of 1 → $N(0, 1)$
 - We can fit this distribution to, for example, 1-year stock returns (change in prices)
 - To assess stock returns in 2 years, we can sum up two (independent) Normal distributions.
 - Mean of the sum distr. equals sum of the means: $\mu = 0 + 0 = 0$
 - σ^2 of the sum is sum of σ_t^2 , so: $\sigma = \sqrt{1+1} = \sqrt{2}$

Wiener Processes / Brownian Motion

Formally, Wiener processes have 2 properties:

- **Property 1:** If z follows a Wiener process, then the change, Δz , during a small period, Δt is:

$$\Delta z = \epsilon \sqrt{\Delta t}$$

where ϵ follows a standard normal distribution $N(0, 1)$

- **Property 2:** The values of Δz for any 2 short intervals of time, Δt , are independent.

From the previous 2 properties, it follows that Δz follows $N(0, \sqrt{\Delta t})$

Thus, Δz , or dz as we will call it later, serves us well to account for uncertainty (change in prices)

- But we also need to account for the level and return of stock prices.

Picture a stock with constant annual return of 10% (μ). Then, if no uncertainty, change in price ΔS :

$$\Delta S = \mu S \Delta t, \quad \text{which as } \Delta t \rightarrow 0, \quad dS = \mu S dt$$

(Geometric) Brownian Motion (GBM)

From the previous expression, we now know how to account for:

- (i) Regular/expected changes in stock price (μ) $\rightarrow$ Drift
- (ii) Uncertainty/unexpected change in prices ($\sigma, \Delta z$) $\rightarrow$ Volatility

- Combining both, we can model changes in stock prices, dS , as:

$$dS = S (\mu dt + \sigma dW_t)$$

where μ and σ are expected return and volatility, and $dW_t = \epsilon \sqrt{dt}$, with $\epsilon \sim \mathcal{N}(0, 1)$

$\Rightarrow$ We refer to this process as **(Geometric) Brownian Motion** $\rightarrow$ We'll see the geometric part next lesson.

- Interestingly, now that we know how to model prices, we can aggregate changes to construct a stock price path:

$$S_1 = S_0 + dS_1; \quad S_2 = S_1 + dS_2; \quad \dots$$

Summary of Lesson 1

In Lesson 1, we have:

- ▶ Introduced a continuous-time framework for option pricing.
- ▶ Covered the basics of Markov and Wiener processes.
- ▶ Learned a way to simulate future stock prices according to GBM process.

⇒ **TO-DO NEXT:** Now, please go to the associated Jupyter Notebook for this lesson to learn how to implement the previous GBM process in Python. This will be the basis for all the remaining pricing in the course (and future courses), so make sure you understand it perfectly.

⇒ In the next lesson, we will continue developing the GBM intuition and introduce Itô calculus to solve the differential equation on dS . This is the first step towards understanding the famous Black-Scholes option pricing model.

Lesson 1 - Markov's property and GBM

February 23, 2026

0.1 DERIVATIVE PRICING

MODULE 4 | LESSON 1

1 MARKOV'S PROPERTY AND GBM

Reading Time	45 minutes
Prior Knowledge	Random numbers, SDE, GBM
Keywords	GBM, Simulation

In Module 4, we will apply the same option-pricing concepts from before to a continuous-time framework. Our first step will be to work on understanding the kind of stochastic process that we will use to model stock prices. In the same spirit as the different paths we constructed in the binomial framework, we are going to build up some “paths” that model future stock price behavior.

All rights reserved WQU WorldQuant University QQQQ

1.1 1. Generating Random Numbers

To start, let's see how we can define random numbers that follow a Wiener process (Brownian motion) in Python. The numpy module `random.randn` will allow us to generate random numbers under a standard normal distribution $N(0, 1)$:

```
[1]: import numpy as np
```

```
X = np.random.randn(100)
X
```

```
[1]: array([ 1.92644656, -0.5763485 , -1.89761512,  0.40822315, -1.00544121,
          -1.47019275,  0.02693397, -0.92053395,  1.4057399 , -1.07154813,
           0.09582104, -0.75818368, -0.60911707,  1.73149838,  0.78256054,
           1.89842701, -0.4202192 , -0.77975919,  0.26669782,  0.20097777,
          -1.24025973,  0.74910208,  0.87321839,  0.38456424,  1.75864664,
          -1.31663311, -0.1485446 ,  1.14260663,  0.71430389, -0.67971365,
           0.61921111, -1.16914453, -1.61423328, -1.02008764, -1.52110675,
          -0.34303566, -0.59192832, -0.19901302, -0.59030062,  2.81497157,
```

```
-0.62053867, -1.2608685 , -0.30802721, 1.6260528 , -1.09462878,
1.61880897, 0.66403817, 1.05425264, -0.52084257, 0.1165173 ,
-1.44582609, 1.07417724, 0.68019161, 0.64944468, 0.30586021,
1.19204757, -0.56659648, 0.29213909, -0.11548633, -0.49160084,
1.64582819, -1.0345072 , 0.61933235, 0.89322921, -0.73824483,
-0.12255432, 1.240963 , -1.07161034, 0.62435358, -0.89342625,
0.31655383, 0.19497874, 1.34972768, 0.19133726, -1.63299837,
-0.23562654, -0.96522542, 1.41251347, 0.89455635, 1.04809883,
-1.0233173 , -0.56420671, 1.19949016, -0.52043217, 1.82140985,
-1.1105755 , 0.44151481, -3.14897293, -1.06263889, -0.67226543,
-1.91955317, 0.4441167 , -0.99528527, 0.3088575 , -0.91414653,
1.10148285, 0.04729031, 0.21066543, 1.19780778, -2.18880907])
```

Obviously, since these are random numbers, every time we execute this code, the output numbers will change. Sometimes, however, it is desirable to always work with the “same” random numbers, especially to draw comparisons between different results and modeling approaches.

Random numbers in any programming language are generated following an algorithm in a not at all random way. These algorithms take a “random” seed as input and generate random numbers from it. Consequently, if we set the initial seed to always be the same value, the outcome of the `np.random.rand(n)` will always be the same!

```
[2]: np.random.seed(1)
X = np.random.randn(100)
X
```

```
[2]: array([ 1.62434536, -0.61175641, -0.52817175, -1.07296862, 0.86540763,
-2.3015387 , 1.74481176, -0.7612069 , 0.3190391 , -0.24937038,
1.46210794, -2.06014071, -0.3224172 , -0.38405435, 1.13376944,
-1.09989127, -0.17242821, -0.87785842, 0.04221375, 0.58281521,
-1.10061918, 1.14472371, 0.90159072, 0.50249434, 0.90085595,
-0.68372786, -0.12289023, -0.93576943, -0.26788808, 0.53035547,
-0.69166075, -0.39675353, -0.6871727 , -0.84520564, -0.67124613,
-0.0126646 , -1.11731035, 0.2344157 , 1.65980218, 0.74204416,
-0.19183555, -0.88762896, -0.74715829, 1.6924546 , 0.05080775,
-0.63699565, 0.19091548, 2.10025514, 0.12015895, 0.61720311,
0.30017032, -0.35224985, -1.1425182 , -0.34934272, -0.20889423,
0.58662319, 0.83898341, 0.93110208, 0.28558733, 0.88514116,
-0.75439794, 1.25286816, 0.51292982, -0.29809284, 0.48851815,
-0.07557171, 1.13162939, 1.51981682, 2.18557541, -1.39649634,
-1.44411381, -0.50446586, 0.16003707, 0.87616892, 0.31563495,
-2.02220122, -0.30620401, 0.82797464, 0.23009474, 0.76201118,
-0.22232814, -0.20075807, 0.18656139, 0.41005165, 0.19829972,
0.11900865, -0.67066229, 0.37756379, 0.12182127, 1.12948391,
1.19891788, 0.18515642, -0.37528495, -0.63873041, 0.42349435,
0.07734007, -0.34385368, 0.04359686, -0.62000084, 0.69803203])
```

You can execute the previous code a few times to check that the random numbers generated are always the same as long as we do not change the seed.

This is a great thing to have when we have to draw comparisons between models or check the performance of our pricing algorithm.

1.2 2. The Geometric Brownian Motion (GBM)

Next, let's see how we can incorporate the random numbers generated in order to simulate our stock price paths under a geometric Brownian motion (GBM) process.

Note that the equations below on dS and S correspond to the same expressions you have in the slides. That is,

$$dS = S_0 (\mu dt + \sigma dW_t)$$

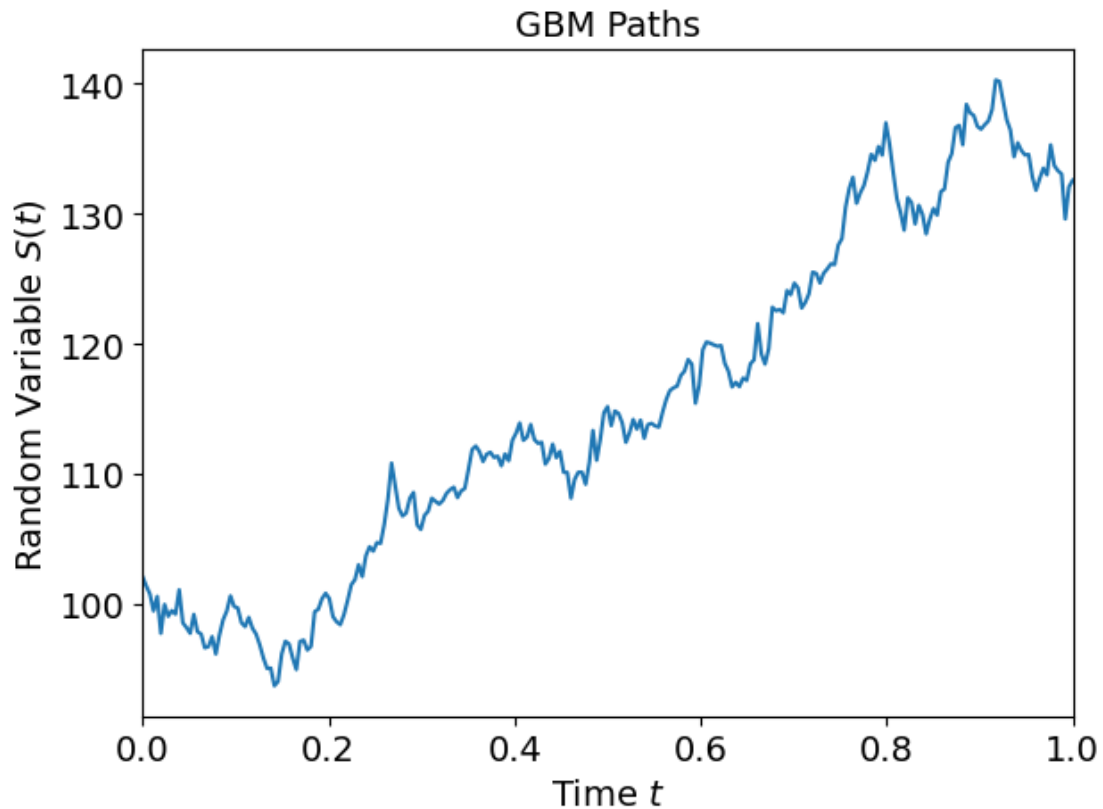
```
[3]: import matplotlib.pyplot as plt

fig = plt.figure()
np.random.seed(1)
T = 1 # years
N = 255 # Number of points, number of subintervals = N-1
dt = T / N # Time step in "years"
t = np.linspace(0, T, N)
M = 1 # Number of paths (Monte Carlo approach)
vol = 0.2
S0 = 100
mu = 0.1 # drift

dS = S0 * (mu * dt + vol * np.sqrt(dt) * np.random.randn(M, N))
S = S0 + np.cumsum(dS, axis=1)

for i in range(M):
    plt.plot(t, S[i, :])

plt.xlabel("Time $t$", fontsize=14)
plt.ylabel("Random Variable $$S(t)$$", fontsize=14)
plt.title("GBM Paths", fontsize=14)
axes = plt.gca()
axes.set_xlim([0, T])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
plt.tight_layout()
plt.show()
```



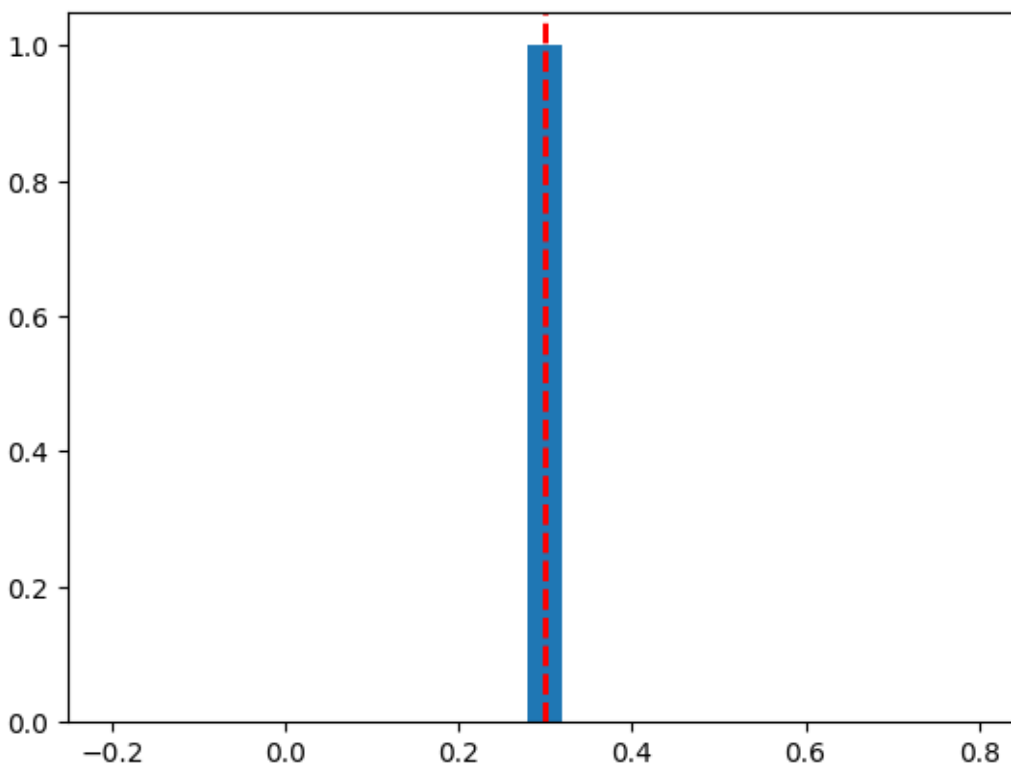
In the previous graphs, you have the different paths generated with the GBM stochastic differential equation (SDE).

Finally, let's explore how the returns from these price paths look in a histogram:

```
[4]: Returns = (S[:, -1] - S[:, 0]) / S[:, 0]
plt.hist>Returns, bins=25)

plt.axvline(np.percentile>Returns, 10), color="r", linestyle="dashed",
↳linewidth=2)
plt.axvline(np.percentile>Returns, 90), color="r", linestyle="dashed",
↳linewidth=2)

plt.show()
```



Once we have the complete distribution of simulated returns, we can observe some characteristics of it. We will get back to this in the future when we start trying to incorporate more real-life features (e.g., jumps in the price) in our modeling equations. For now, a very simple analysis can be done with the use of risk metrics like Value at Risk (VaR). The VaR will measure the maximum drop in the returns given a certain level of confidence (in terms of the probability distribution):

```
[5]: print(np.percentile>Returns, 5)) # Value at Risk at 5%
```

```
0.2987108325858604
```

In this case, given the parameters of the GBM equation and simulated prices and returns, we could say that with 95% probability the stock will not drop more than 22% in a given day.

1.3 3. Conclusion

In this lesson, we have seen how to generate random numbers and simulate stock price paths using the GBM equation. Let's now go to the next lesson, in which we will see how to analytically solve the previous SDE for the GBM process using Itô calculus.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 4, Section 4.2
- **Estimated reading time:** 15 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 2

ITÔ'S LEMMA AND BLACK-SCHOLES MODEL

Derivative Pricing - Module 4



Outline

- ▶ Log-normal property of stock prices
- ▶ Itô's Lemma and the solution to GBM SDE
- ▶ Black-Scholes model

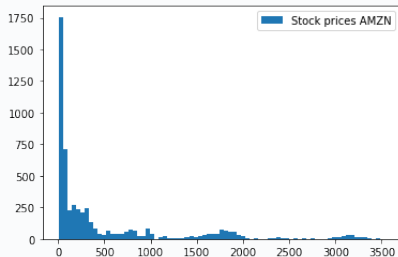
Log-Normal Property of Stock Prices

In order to analytically solve the SDE in the previous lesson, we are going to make use of two things:

- (i) Log-normal property of stock prices
- (ii) Itô's Lemma to solve the SDE

Let's explore the first of these $\Rightarrow$ Is it realistic to assume stock prices follow a log-normal distribution?

Go to the first part of the Jupyter Notebook accompanying this lesson to see this:



Itô calculus for GBM

Conveniently for us, a log-normal transform of a log-normally distributed variable will follow a normal distribution.

- By taking $S_t = \ln(S_t)$ and applying **Itô's Lemma**, we can get a closed-form solution for S_t from our SDE:

$$\ln \frac{S_t}{S_0} = \left(\mu - \frac{\sigma^2}{2} \right) t + \sigma W_t,$$

$$\ln S_t \sim \mathcal{N} \left(\ln S_0 + \left(\mu - \frac{\sigma^2}{2} \right) t, \sigma^2 t \right)$$

which yields:

$$S_t = S_0 e^{\left(\left(\mu - \frac{\sigma^2}{2} \right) t + \sigma W_t \right)}$$

where,

$$E(S_t) = S_0 e^{\mu t}$$

$$\text{Var}(S_t) = S_0^2 e^{2\mu t} (e^{\sigma^2 t} - 1)$$

Black-Scholes-Merton Model

The last SDE gave rise to one of the most important option pricing models ever: **Black-Scholes**.

The model departs from some important assumptions (some will be relaxed in the future):

- (i) Prices follow a GBM with μ and σ constant.
 - (ii) Perfect capital markets: No taxes, transaction costs, no dividends, absence of arbitrage, etc.
 - (iii) Risk-free rate is constant and equal for all maturities.
- ▶ Next, apply the same intuition as in binomial setting: No-arbitrage and riskless portfolio replica!
 - ▶ This yields the following **Black-Scholes-Merton SDE** (see additional references for proof):

$$\frac{\partial f}{\partial t} + rS \frac{\partial f}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 f}{\partial S^2} = rf$$

for f being the claim associated with any derivative instrument.

- ▶ Importantly, note how the previous equation only depends on $r \rightarrow$ Risk-neutral valuation

Black-Scholes Formulas for Option Pricing

Using the Black-Scholes-Merton SDE, we can get closed-form solutions for call and put option prices.

- The formulas are the following:

$$\text{Call option} \rightarrow c = S_0 \mathcal{N}(d_1) - Ke^{-rT} \mathcal{N}(d_2)$$

$$\text{Put option} \rightarrow p = Ke^{-rT} \mathcal{N}(-d_2) - S_0 \mathcal{N}(-d_1)$$

where $\mathcal{N}()$ is the cumulative density function (CDF) of a standard normal and:

$$d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}$$

$$d_2 = \frac{\ln(S_0/K) + (r - \sigma^2/2)T}{\sigma\sqrt{T}} = d_1 - \sigma\sqrt{T}$$

- Again, only discounting on $r \rightarrow$ Risk-neutral valuation

Greeks in Black-Scholes

In the final part of the lesson, we are going to deal with the **Greeks in a Black-Scholes model**.

We can interpret Greeks as **the sensitivity of option price to different variables** → remember Delta?

- Sensitivity to underlying stock price (**Delta**):

$$\text{Call Option } \Delta = \frac{\partial C}{\partial S} = \mathcal{N}(d_1); \quad \text{Put Option } \Delta = \frac{\partial P}{\partial S} = \mathcal{N}(d_1) - 1$$

- Sensitivity to changes in underlying stock price (**Gamma**):

$$\Gamma = \frac{\partial^2 V}{\partial S^2} = \frac{\mathcal{N}'(d_1)}{S\sigma\sqrt{T-t}}$$

- Sensitivity to volatility (**Vega**):

$$\nu = \frac{\partial^2 V}{\partial \sigma} = S\mathcal{N}'(d_1)\sqrt{T-t}$$

Greeks in Black-Scholes (con.)

- Sensitivity to time (**Theta**):

$$\text{Call Option } \Theta = \frac{\partial C}{\partial t} = -\frac{SN'(d_1)\sigma}{2\sqrt{T-t}} - rKe^{-r(T-t)}\mathcal{N}(d_2)$$

$$\text{Put Option } \Theta = \frac{\partial P}{\partial t} = -\frac{SN'(d_1)\sigma}{2\sqrt{T-t}} + rKe^{-r(T-t)}\mathcal{N}(-d_2)$$

- Sensitivity to risk-free rate (**Rho**):

$$\text{Call Option } \rho = \frac{\partial C}{\partial r} = K(T-t)e^{-r(T-t)}\mathcal{N}(d_2)$$

$$\text{Put Option } \rho = \frac{\partial P}{\partial r} = -K(T-t)e^{-r(T-t)}\mathcal{N}(-d_2)$$

⇒ You can consult additional references to see more on the derivation of each of these.

Summary of Lesson 2

In Lesson 2, we have looked at:

- ▶ Log-normality of stock prices
- ▶ Derivation of GBM process for stock prices using Itô's Lemma
- ▶ Black-Scholes-Merton SDE and option pricing model
- ▶ Greeks (sensitivities) in Black-Scholes model

⇒ **TO-DO NEXT:** Now, please go to the associated Jupyter Notebook for this lesson to learn how to implement the Black-Scholes option pricing model in Python. Also, you have a working example of the different Greeks there. Lastly, please make sure that you **check the additional material in this lesson** to ensure that you have a comprehensive understanding of how to derive the previous formulas (Itô's Lemma, GBM, Black-Scholes, etc.)

⇒ In the next lesson, we will see the application of a previously introduced method, **Monte Carlo**, for pricing options under the Black-Scholes framework.

DERIVATIVE PRICING

MODULE 4 | LESSON 2

ITO'S LEMMA AND BLACK-SCHOLES MODEL

Reading Time	65 minutes
Prior Knowledge	GBM, Black-Scholes
Keywords	Ito's lemma, GBM, Black-Scholes, Greeks

*In this lesson, we will work on developing the intuition and main code to implement the **Black-Scholes** option pricing model.*

All the closed-form equations used here are developed intuitively in the slides/video for this lesson and with more mathematical detail in the additional readings and videos for the lesson. Make sure you check those before continuing on.

As usual, let's begin by importing the necessary libraries:

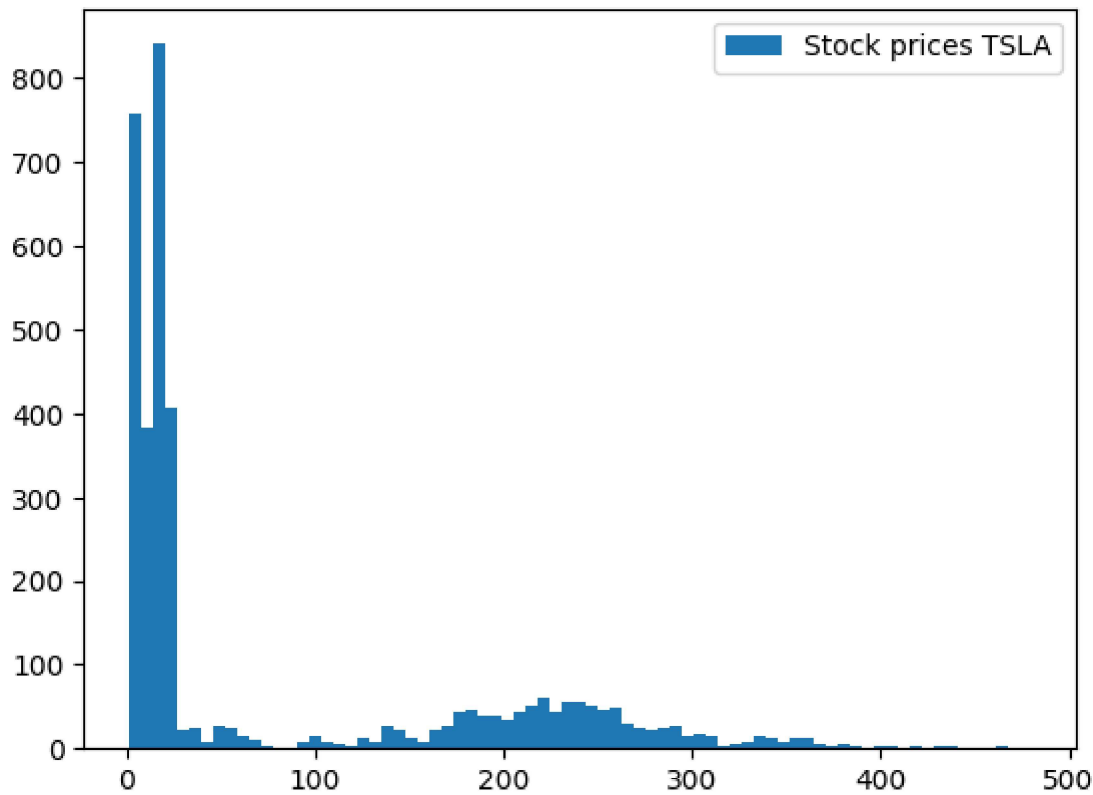
```
In [1]:import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import lognorm, norm
import pandas as pd
```

1. The Log-Normal Property of Stock Prices

Let's start by exploring the historical distribution of stock prices. We will come back to this later on, but at this point, it will serve to illustrate the reason (or simplification made) for using log-normal distributions.

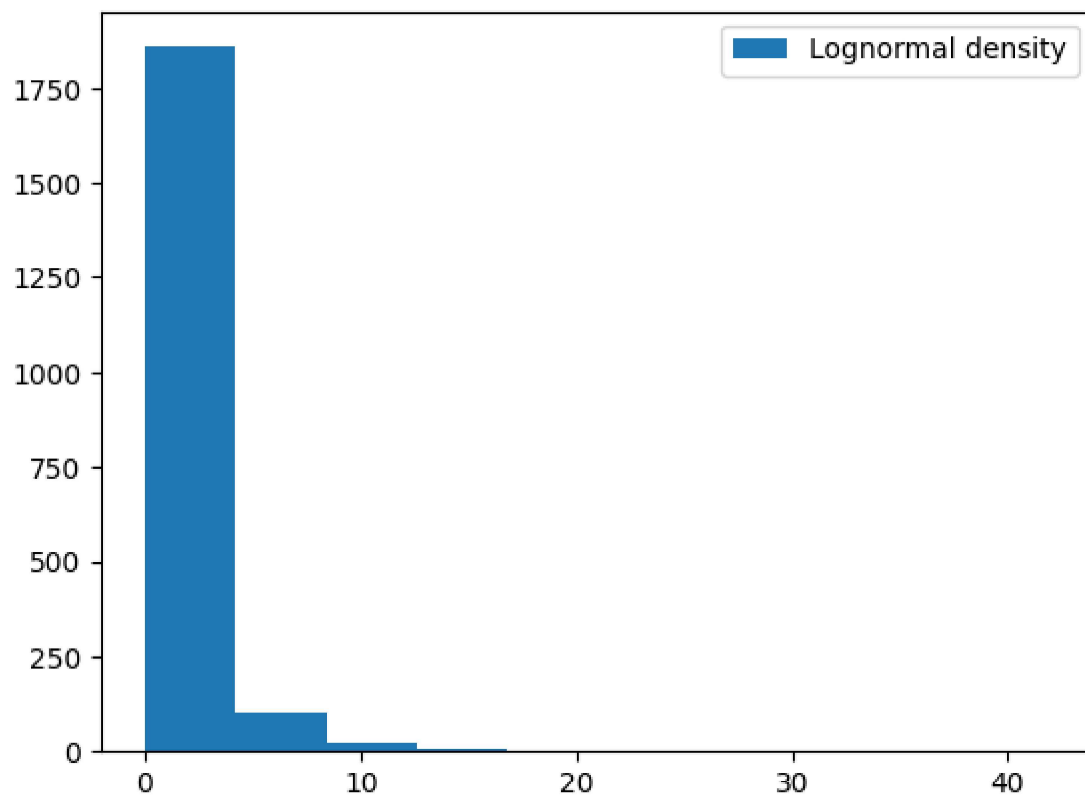
To do that, let's quickly download some historical price data from TSLA stock.

```
In [2]:tsla = pd.read_csv('TSLA_wqu_data.csv', parse_dates=
In [3]:plt.hist(tsla['close'], bins=75, label='Stock prices TSLA
plt.legend()
plt.show()
```



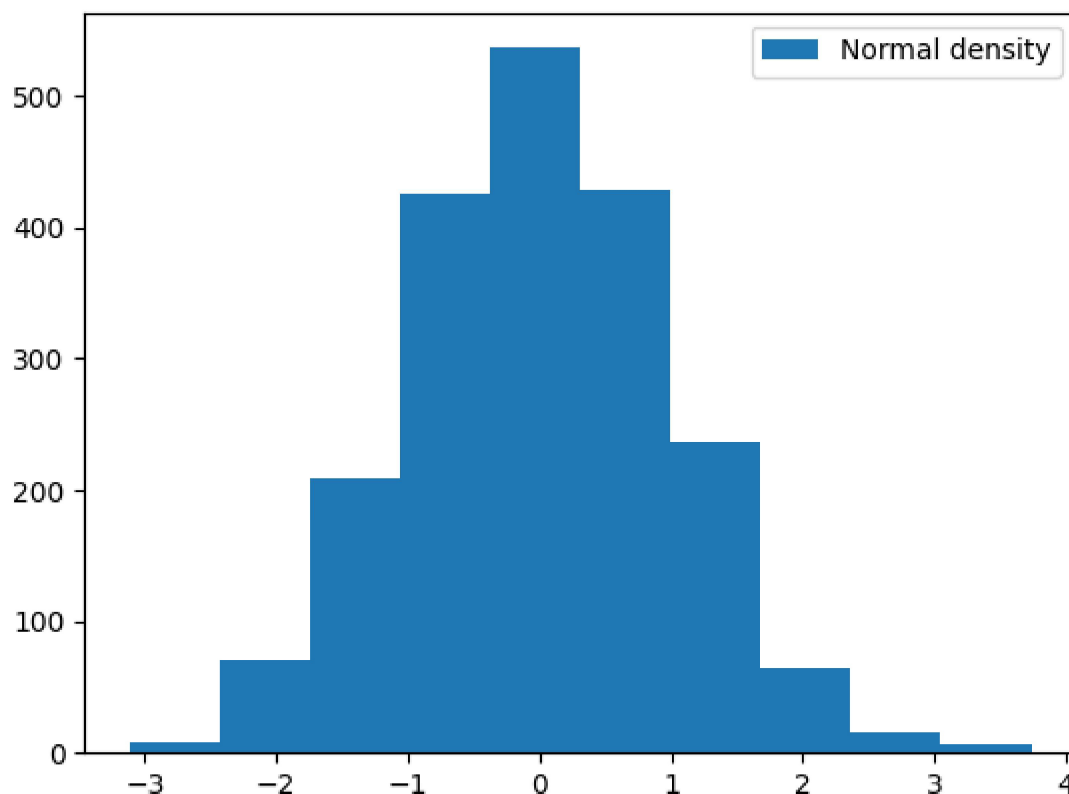
As you can see, this somewhat resembles a log-normal distribution.

```
In [4]: r = lognorm.rvs(1, size=2000)
plt.hist(r, label="Lognormal density")
plt.legend()
plt.show()
```



Now, there may be some arguments in favor and against this resemblance to log-normal distributions (e.g., what happens with large shocks?). However, for now, we are going to assume this log-normality of stock prices, which implicitly means that we can model changes in stock prices (returns) using logarithms (Ln). Conveniently for us, the log-transform of a log-normally distributed variable will follow a normal distribution.

```
In [5]: s = np.log(r)
plt.hist(s, label="Normal density")
plt.legend()
plt.show()
```



2. Pricing a Call Option with Black-Scholes

Now that we have investigated the intuition behind the log-normal assumption for stock prices, let's proceed to a complete pricing example using the **Black-Scholes model** and the equations developed in the slides.

Let's start by building a function (`bs_call_price`) that returns the value of a call option under the Black-Scholes model. Remember:

$$S_c = S_0 \mathcal{N}(d_1) - K e^{-rT} \mathcal{N}(d_2)$$

```
In [6]: def bs_call_price(S, r, sigma, t, T, K):
    ttm = T - t

    if ttm < 0:
        return 0.0
    elif ttm == 0.0:
        return np.maximum(S - K, 0.0)
```

```

vol = sigma * np.sqrt(ttm)

d_minus = np.log(S / K) + (r - 0.5 * sigma**2) * ttm
d_minus /= vol

d_plus = d_minus + vol

res = S * norm.cdf(d_plus)
res -= K * np.exp(-r * ttm) * norm.cdf(d_minus)

return res

```

Now, let's check how our function performs to price a call option with:

- $S_0 = 100$
- $r = 0$
- $\sigma = 20\%$
- $t = 0$
- $T = 2$
- $K = 105$

```
In [7]: print(bs_call_price(100.0, 0.0, 0.2, 0, 2.0, 105.0))
```

```
9.19735064929452
```

3. Greeks

Next, let's implement the calculation of the **Greeks**, as seen from the closed-form formulas in the slides. At this point, let's just think of the Greeks as sensitivities of option prices to movements in different variables of interest. In the future, we will dig deeper into this part in order to use Greeks to **hedge away the different risks we may be facing in a portfolio**.

```

In [8]: # Black-Scholes model in Python
import numpy as np
import scipy.stats as ss

# Data for input in Black-Scholes formula:

T = 2.0 # supposed in years. It is not the maturity, but
S = 100.0
K = 105.0
r = 0
vol = 0.20 # supposing it is annual
option_type = "C" # for the put insert 'P'
# dividend yield assumed to be 0

# Compute d1 and d2
d1 = (np.log(S / K) + (r + 0.5 * vol**2) * T) / (vol * np
d2 = d1 - vol * np.sqrt(T)

if option_type in ["C", "P"]:
    if option_type in ["C"]:
        Opt_Price = S * ss.norm.cdf(d1) - K * np.exp(-r *

```

```

Delta = ss.norm.cdf(d1)
Gamma = ss.norm.pdf(d1) / (S * vol * np.sqrt(T))
Vega = S * ss.norm.pdf(d1) * np.sqrt(T)
Theta = -(S * ss.norm.pdf(d1) * vol) / (2 * np.sqrt
-r * T
) * ss.norm.cdf(d2)
Rho = K * T * np.exp(-r * T) * ss.norm.cdf(d2)
else:
Opt_Price = K * np.exp(-r * T) * ss.norm.cdf(-d2)
Delta = -ss.norm.cdf(-d1)
Gamma = ss.norm.pdf(d1) / (S * vol * np.sqrt(T))
Vega = S * ss.norm.pdf(d1) * np.sqrt(T)
Theta = -(S * ss.norm.pdf(d1) * vol) / (2 * np.sqrt
-r * T
) * ss.norm.cdf(-d2)
Rho = -K * T * np.exp(-r * T) * ss.norm.cdf(-d2)
else:
Opt_Price = "Error: option type incorrect. Choose P

print("Option price = {}".format(Opt_Price))
print("Delta = {}".format(Delta))
print("Gamma = {}".format(Gamma))
print("Vega = {}".format(Vega))
print("Theta = {}".format(Theta))
print("Rho = {}".format(Rho))

```

```

Option price = 9.197350649294513
Delta = 0.4876036978454982
Gamma = 0.014097929791127266
Vega = 56.39171916450907
Theta = -2.819585958225453
Rho = 79.1260382705106

```

4. Conclusion

In this lesson, we have looked at the lognormal property of stock prices and covered the basics of option pricing under the Black-Scholes model, including the different Greeks.

Now that you know the basics of Black-Scholes pricing and the lognormal property of stock prices, proceed to the next lesson in order to work more on the Black-Scholes option pricing model and related features.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 4, Section 4.4.3
- **Estimated reading time:** 20 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 3

BLACK-SCHOLES AND MONTE CARLO METHODS

Derivative Pricing - Module 4



Outline

- ▶ Risk-neutral valuation in the Black-Scholes world: Price dynamics?
- ▶ Monte Carlo methods for option pricing
- ▶ Convergence between Black-Scholes and Monte Carlo

Risk-neutral valuation in the Black-Scholes world

In the last lesson, we derived the option pricing formulas from the Black-Scholes model under certain price dynamics for stock prices:

$$dS = S (\mu dt + \sigma dW_t)$$

However, as we mentioned a few times already, we are working in a risk-neutral world:

- ▶ Where we work with risk-neutral problems
 - ▶ Where the return earned by an asset is r , the risk-free rate
- ⇒ Remember that these two changes offset each other.

Hence, to work in a risk-neutral setting, our Brownian Motion SDE will be:

$$dS = S (rdt + \sigma dW_t), \text{ discretized by } \rightarrow S_T = S_t e^{\left(\left(r - \frac{\sigma^2}{2}\right)(T-t) + \sigma\sqrt{T-t}Z\right)}$$

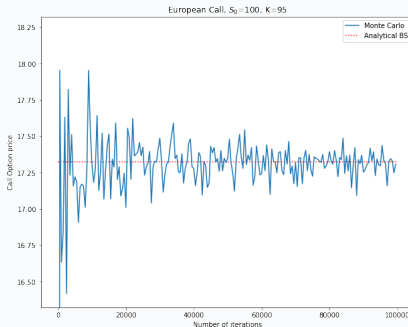
with $Z \sim \mathcal{N}(0, 1)$

Monte Carlo methods and convergence with Black-Scholes

You are already familiar with Monte Carlo methods and their powerful applications for option pricing.

In this lesson, we are going to revisit this technique to verify that, under the risk-neutral world price dynamics in the Black-Scholes world, we are able to replicate the results from the closed-form analytical solution.

- Now, go to the Jupyter Notebook associated with this lesson to check this out.



Summary of Lesson 3

In Lesson 3, we've looked at:

- ▶ Monte Carlo methods in the Black-Scholes world
- ▶ Convergence between Monte Carlo method and Black-Scholes analytical price

⇒ **TO-DO NEXT:** Now, please go to the associated Jupyter Notebook for this lesson if you haven't done so yet.

⇒ In the next lesson, we will see how to apply all these techniques to a different underlying asset price process.

Lesson 3 - Black-Scholes and Monte Carlo Methods

February 23, 2026

0.1 DERIVATIVE PRICING

MODULE 4 | LESSON 3

1 BLACK-SCHOLES AND MONTE CARLO METHODS

Reading Time	55 minutes
Prior Knowledge	Black-Scholes, GBM, Monte-Carlo
Keywords	Monte-Carlo, Black-Scholes, Simulation, Convergence between methods

In this lesson, we will expand our knowledge of the Black-Scholes framework by considering a Monte Carlo approach to pricing options under an SDE. Specifically, we will verify that Monte Carlo methods applied on the appropriate SDE converge with the analytical Black-Scholes model price.

As always, let's first import some libraries we will need down the road:

```
[1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

%matplotlib inline
import numpy.random as npr
from scipy.stats import norm
```

1.1 1. Monte Carlo Method

An alternative way to compute the call option price is to use Monte Carlo methods. This, obviously, makes little sense now (we do have a closed-form solution, right?), but it will come in handy when we move to more complex frameworks. The starting point is taking the Brownian motion equation for the underlying stock, S_t , in the risk-neutral world:

$$dS = S r dt + S \sigma dz$$

This equation can be discretized by a Euler scheme. By doing this, we obtain the stock price at a future date given by:

$$S_T = S_t e^{((r - \frac{1}{2}\sigma^2)(T-t) + \sigma\sqrt{T-t}z)},$$

where z is a stochastic variable following $\mathcal{N}(0, 1)$.

We can simulate the process above using the Monte Carlo method. As an example, we set $r = 0.06$, $\sigma = 0.3$, $T - t = 1$, $S_t = 100$. We make $I = 100000$ iterations and choose $M = 100$ intervals for the time interval discretization:

```
[2]: r = 0.06
sigma = 0.3
T = 1.0
S0 = 100

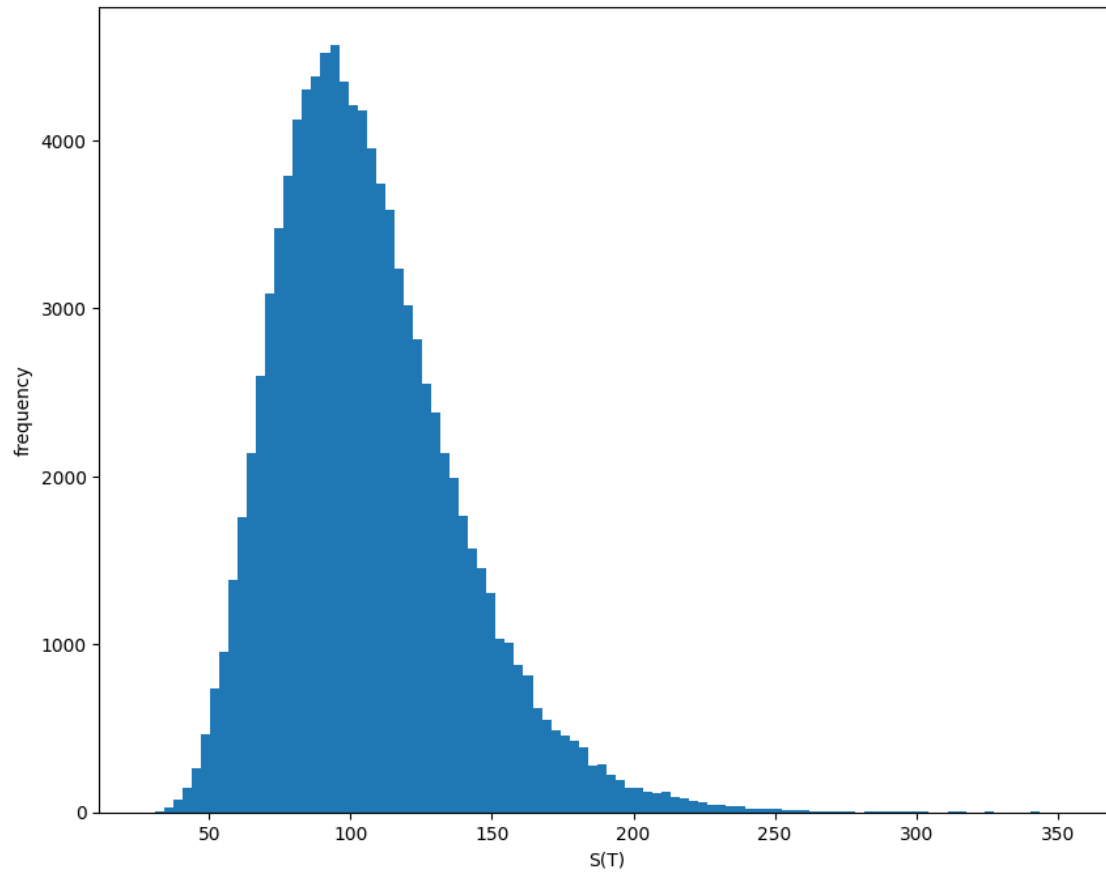
Ite = 100000
M = 100

dt = T / M

S = np.zeros((M + 1, Ite))
S[0] = S0
for t in range(1, M + 1):
    S[t] = S[t - 1] * np.exp(
        (r - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * npr.
        ↪ standard_normal(Ite)
    )
```

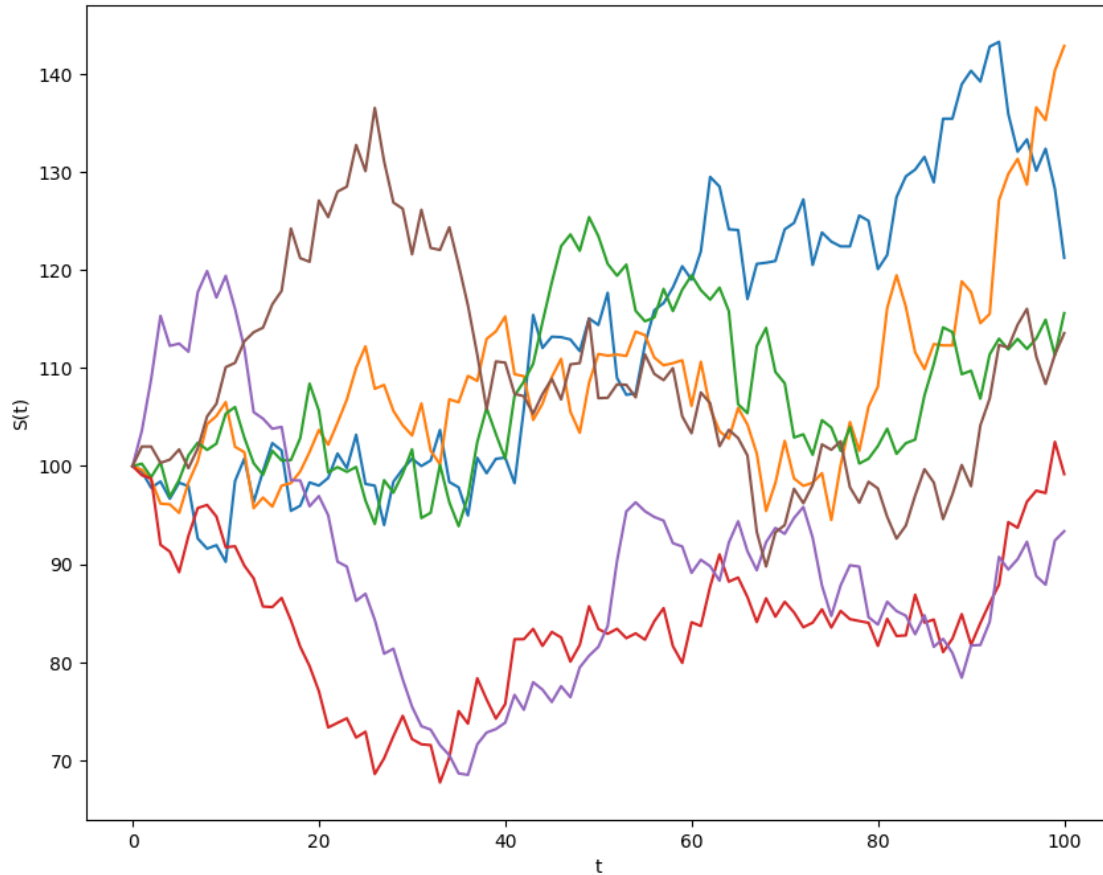
Now, we have the simulated values for stock prices, S_t . According to the theoretical setting, these prices should follow a log-normal distribution. Let's check and see.

```
[3]: plt.figure(figsize=(10, 8))
plt.hist(S[-1], bins=100)
plt.xlabel("S(T)")
plt.ylabel("frequency")
plt.show()
```



Yes! S_t follows a log-normal distribution. Let's also visualize a few of the simulated paths under the previous parameters to get a feeling of how stock prices move in each path:

```
[4]: plt.figure(figsize=(10, 8))  
plt.plot(S[:, :6])  
plt.xlabel("t")  
plt.ylabel("S(t)")  
plt.show()
```



Now, it is time to implement our Monte Carlo algorithm. We will construct a function, `bs_call_mc`, that calculates the price, according to the Monte Carlo method, of a European call option. Remember the steps we are doing in the code below:

- Simulate stock price paths according to SDE dynamics.
- Calculate the terminal option payoff for each path.
- Obtain the average expected payoff at maturity, T .
- Discount the average expected payoff to time t .

```
[5]: def bs_call_mc(S, K, r, sigma, T, t, Ite):
    data = np.zeros((Ite, 2))
    z = np.random.normal(0, 1, [1, Ite])
    ST = S * np.exp((T - t) * (r - 0.5 * sigma**2) + sigma * np.sqrt(T - t) * z)
    data[:, 1] = ST - K

    average = np.sum(np.amax(data, axis=1)) / float(Ite)

    return np.exp(-r * (T - t)) * average
```

Let's see the pricing power of the previous function with a European call option with the following characteristics:

- $S_0 = 100$
- $K = 95$
- $r = 0.06$
- $\sigma = 0.3$
- $T = 1$ year
- $t = 0$ (present day)
- Number of iterations $I = 100000$

```
[6]: print("Monte Carlo Price:", bs_call_mc(100, 95, 0.06, 0.3, 1, 0, 100000))
```

Monte Carlo Price: 17.26791095314757

1.2 2. Convergence of Monte Carlo to Black-Scholes

Next, let's check the price that our analytical pricing formula (the one we implemented in the last lesson) yields.

```
[7]: def bs_call_price(S, r, sigma, t, T, K):
    ttm = T - t

    if ttm < 0:
        return 0.0
    elif ttm == 0.0:
        return np.maximum(S - K, 0.0)

    vol = sigma * np.sqrt(ttm)

    d_minus = np.log(S / K) + (r - 0.5 * sigma**2) * ttm
    d_minus /= vol

    d_plus = d_minus + vol

    res = S * norm.cdf(d_plus)
    res -= K * np.exp(-r * ttm) * norm.cdf(d_minus)

    return res
```

```
[8]: print("BS Analytical Price:", bs_call_price(100, 0.06, 0.3, 0, 1, 95))
```

BS Analytical Price: 17.323563283257634

We observe that the Monte Carlo price (17.42) is actually very close to the Black-Scholes analytical price (17.32). As usual, the convergence to the true analytical price will depend on the number of simulations. Thus, once again, we face the trade-off between computational costs and accuracy.

Next, let's explore how quickly (or slowly) Monte Carlo estimates converge with Black-Scholes analytical prices:

```
[9]: df = pd.DataFrame(columns=["Iter", "BSc"])
```

```

for i in range(1, 100000, 500):
    df = pd.concat([df, pd.DataFrame(
        [{"Iter": i, "BSc": bs_call_mc(100,95,0.06,0.3,1,0,i)}]]), ignore_index=True
    )

plt.figure(figsize=(10, 8))

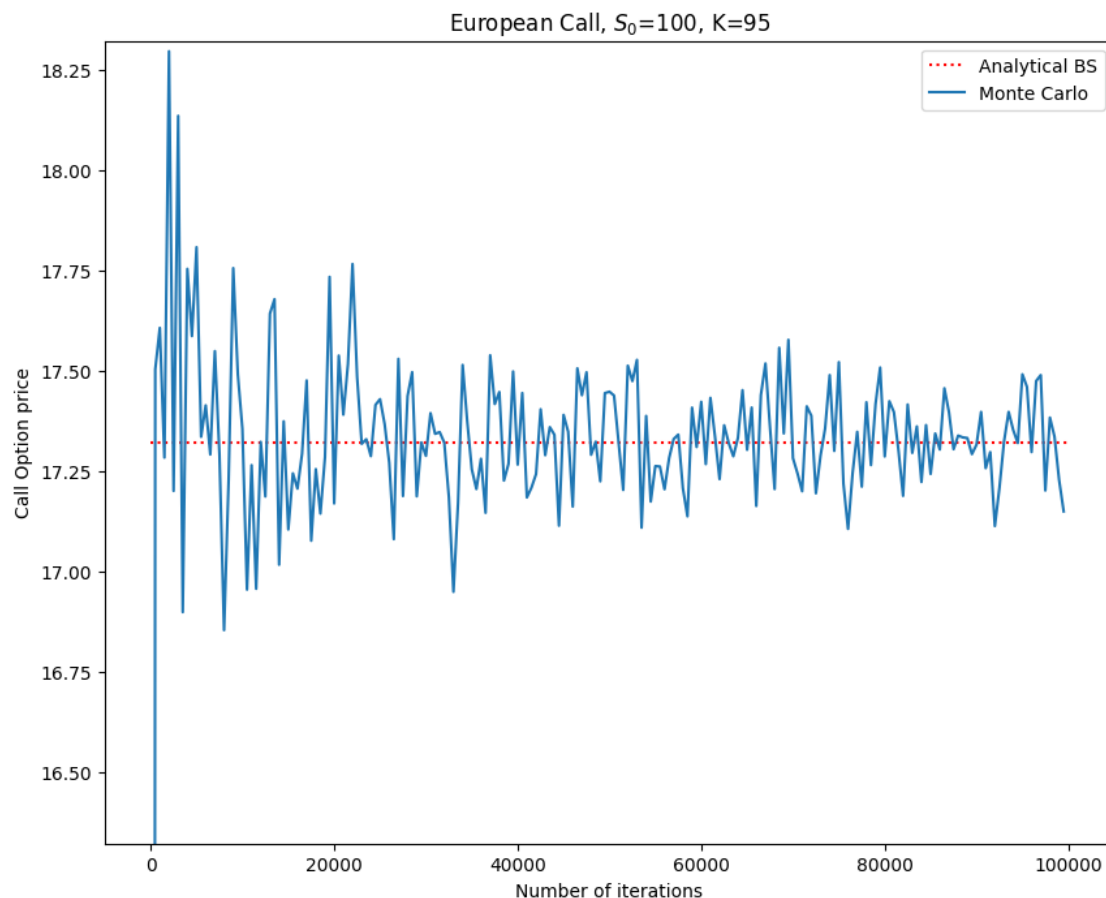
plt.hlines(
    bs_call_price(100, 0.06, 0.3, 0, 1, 95),
    xmin=0,
    xmax=100000,
    linestyle="dotted",
    colors="red",
    label="Analytical BS",
)
plt.plot(df.set_index("Iter"), lw=1.5, label="Monte Carlo")

plt.title("European Call, $S_0$=100, K=95")
plt.xlabel("Number of iterations")
plt.ylabel("Call Option price")
plt.ylim(
    bs_call_price(100, 0.06, 0.3, 0, 1, 95) - 1,
    bs_call_price(100, 0.06, 0.3, 0, 1, 95) + 1,
)
plt.legend()
plt.show()

```

/tmp/ipykernel_18/2310393175.py:4: FutureWarning: The behavior of DataFrame concatenation with empty or all-NA entries is deprecated. In a future version, this will no longer exclude empty or all-NA columns when determining the result dtypes. To retain the old behavior, exclude the relevant entries before the concat operation.

```
df = pd.concat([df, pd.DataFrame(
```



Can you do the same thing for a put option?

1.3 3. Conclusion

Now you know how to implement Monte Carlo methods for the simulation of underlying stock price evaluation in the Black-Scholes world. In the next lesson, we will see that Monte Carlo methods are useful for simulating not only stock prices but also other kinds of assets (such as interest rates) given the appropriate model dynamics.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 4, Section 4.4.4
- **Estimated reading time:** 25 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 4

SIMULATING INTEREST RATES: VASICEK MODEL

Derivative Pricing - Module 4



Outline

- ▶ Vasicek model for interest rates
- ▶ Using Monte Carlo techniques to simulate interest rates under the Vasicek model

Vasicek model (1977)

So far, we have focused on modeling stock price behavior. However, the same intuition can be applied to other types of SDE that capture the behavior of other types of assets.

Specifically, in this lesson, we'll look at how we can model interest rates using the Vasicek (1977) model:

- ▶ **The Vasicek (1977) model** has the following **mean-reverting risk-neutral process** for interest rates, r_t :

$$dr_t = k(\theta - r_t)dt + \sigma dZ_t$$

where $dZ_t = \sqrt{dt}z$, with $z \sim \mathcal{N}(0, 1)$. The different non-negative parameters proxy for:

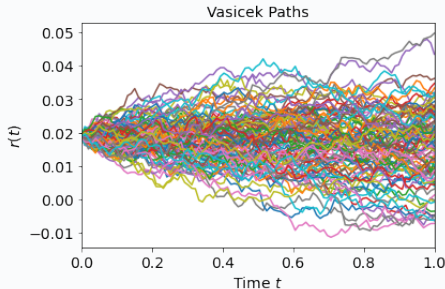
- ▶ θ is long-term mean of interest rates. All paths will evolve around this level.
- ▶ k is the speed of reversion, meaning how quickly rates revert to θ .
- ▶ σ is the instantaneous volatility for interest rates.
- ▶ $\frac{\sigma^2}{2k}$ can be defined as long-term variance.

Simulating interest rates under Vasicek

As we did in previous lessons, given the SDE defined before, we can (under certain parameters) simulate the different paths that interest rates can take in the future.

This, once again, illustrates the power of Monte Carlo techniques once we have an expression for the evolution of the asset.

$$dr_t = k(\theta - r_t)dt + \sigma dZ_t, \text{ so, given an initial } r_0 \rightarrow r_1 = r_0 + dr_1; \dots$$



Summary of Lesson 4

In Lesson 4, we have looked at:

- ▶ Vasicek Model for interest rates
- ▶ Simulating interest rates using Vasicek's SDE

⇒ **TO-DO NEXT:** Now, please go to the associated Jupyter Notebook for this lesson to see how to implement the Vasicek model in Python and the simulation of different paths for interest rates.

⇒ In the next module, we will start looking at the flaws associated with modeling strategies based on SDEs.

Lesson 4 - Simulating Interest Rates Vasicek Model

February 23, 2026

0.1 DERIVATIVE PRICING

MODULE 4 | LESSON 4

1 SIMULATING INTEREST RATES: VASICEK MODEL

Reading Time	40 minutes
Prior Knowledge	Black-Scholes, GBM, Simulation
Keywords	Vasicek, Interest rates model, Mean-reversion

—All rights reserved WQU WorldQuant University QQQQ

*In this lesson, we will tackle the use of stochastic differential equations (SDE) for simulating the behavior of other types of assets. Specifically, we will focus on simulating interest rates using the SDE associated with the **Vasicek (1977) model**.*

As always, let's first import some libraries we will need down the road:

```
[1]: import matplotlib.pyplot as plt
import numpy as np
```

1.1 1. Vasicek (1977) Model

In Vasicek's model, the risk-neutral process for interest rates is:

$$dr = k(-r)dt + \sigma dz$$

where $dz = \sqrt{dt}z$, with $z \sim \mathcal{N}(0, 1)$.

Let's build a function for this SDE:

```
[2]: def vasicek(r0, K, theta, sigma, T, N, M):
    dt = T / N
    rates = np.zeros((N, M))
    rates[0, :] = r0
    for j in range(M):
        for i in range(1, N):
            dr = (
                K * (theta - rates[i - 1, j]) * dt
```

```

        + sigma * np.sqrt(dt) * np.random.normal()
    )
    rates[i, j] = rates[i - 1, j] + dr
return rates

```

Now, we can use the same intuition of the Monte Carlo methods we used before for the Black-Scholes SDE to simulate interest rates given a current level, r_0 .

```

[3]: M = 100  # Number of paths for MC
     N = 100  # Number of steps
     T = 1.0  # Maturity
     r0 = 0.01875
     K = 0.20
     theta = 0.01
     sigma = 0.012
     t = np.linspace(0, T, N)

     rates = vasicek(r0, K, theta, sigma, T, N, M)
     rates.shape

```

```

[3]: (100, 100)

```

We have just simulated interest rates for 100 paths and 100 steps.

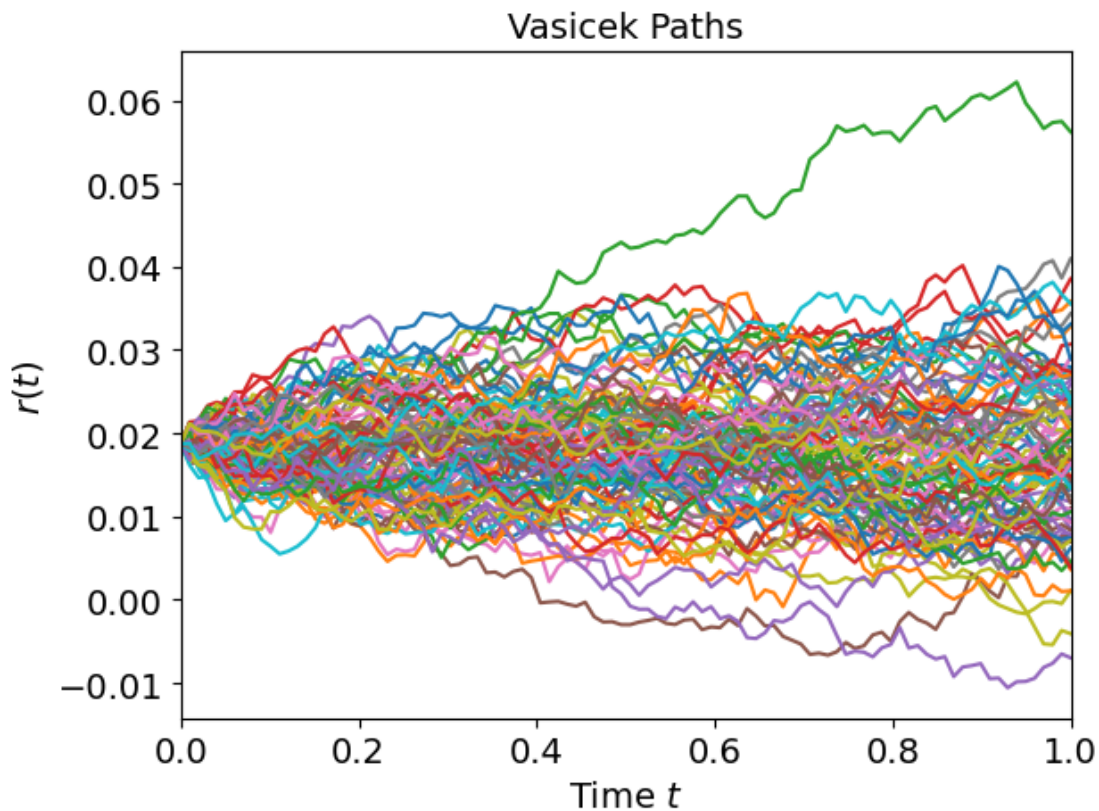
Let's see how these look:

```

[4]: for j in range(M):
     plt.plot(t, rates[:, j])

     plt.xlabel("Time $t$", fontsize=14)
     plt.ylabel("$r(t)$", fontsize=14)
     plt.title("Vasicek Paths", fontsize=14)
     axes = plt.gca()
     axes.set_xlim([0, T])
     plt.xticks(fontsize=14)
     plt.yticks(fontsize=14)
     plt.tight_layout()
     plt.show()

```



1.2 2. Conclusion

In this lesson, you have seen even more of the power of Monte Carlo techniques. As long as we have an expression (e.g., an SDE) that models the behavior of the underlying asset, we can apply this technique to all different kinds of assets, not just equities!

In the next module, we will look at how we can adapt the different models (i.e., SDEs) so that they are able to incorporate the different features of stock prices and returns.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.